



中国科学技术大学
University of Science and Technology of China

Hedera: Dynamic Flow Scheduling for Data Center Networks

Mohammad Al-Fares, Sivasankar Radhakrishnan, et al.
NSDI 2010

授课教师：赵功名
中国科大计算机学院
2026年秋·高级计算机网络

Outline

I. Introduction

II. Background

III. Architecture

IV. Estimation and Scheduling

V. Implementation

VI. Evaluation

VII. Review

Introduction

引言概述

数据中心规模与应用带宽需求不断增长，网络需要高效利用多条路径。本节先分析工作负载、兼容性和虚拟化带来的挑战，再说明现有转发方式的局限，进而引出 Hedera 动态流调度系统

宏观背景铺垫

- DC 规模增长
- 应用带宽需求增长
- 多路径利用挑战

具体问题分析

- 负载不可预测
- 兼容性约束
- 虚拟化影响

现有技术局限

引入本文解决方案

Introduction: The core challenges of DC networks

数据中心的规模与带宽挑战

- 背景：数据中心扩展至数万台主机，科学计算、Web 搜索和 MapReduce 等应用使**集群内（intra-cluster）带宽需求持续增长**
- 核心问题：网络需支持潜在的 **All-to-All 通信**。多根树拓扑通过多条路径突破交换机端口密度限制，关键是**如何充分利用这些路径**
- All-to-All 通信：每个节点向**所有**其他节点发送数据片段，并接收它们发来的数据，最终完成节点之间的数据交换

Introduction: The core challenges of DC networks

云应用的三个特性使网络设计复杂化

- 工作负载不可预测：流量随时间和位置变化，静态分配难以持续匹配需求
- 兼容性要求：应用运行在通用操作系统上，网络需在不修改现有软件或协议的前提下提供高带宽
- 虚拟化影响：应用实例可能分散在不同机架，跨机架通信容易暴露传统拓扑的带宽瓶颈

因此，网络需要**动态适应**流量变化，同时保持对现有应用的兼容

Introduction: The core challenges of DC networks

现有协议与拓扑的不匹配

- 传统协议局限：传统企业网络的通信对象相对稳定、可选路径较少。数据中心则依靠大量并行路径扩展容量，原有转发方式难以充分发挥拓扑优势
- 多根树（Multi-rooted Tree）的潜力与问题：商用交换机端口密度有限，多根树提供多条主机间路径。关键是合理分配流量、避免链路过载；ECMP 的静态哈希仍可能导致碰撞和带宽损失

Introduction: The core challenges of DC networks

ECMP的局限性

- 什么是 ECMP（等价多路径路由）：按流哈希选择一条等价路径，将不同流分散到多条路径，并保持同一流的路径稳定
 - ECMP 问题：**流哈希静态映射**不感知流大小和当前链路负载，多个大流可能落到同一出口，形成可避免的瓶颈
 - Hedera 的提出：收集大流信息，估计带宽需求并动态选择路径，通过交换机重路由提升**二分带宽**
- 全局视角：识别并协调交换机本地决策难以发现的下游瓶颈

Outline

I. Introduction

II. Background

III. Architecture

IV. Estimation and Scheduling

V. Implementation

VI. Evaluation

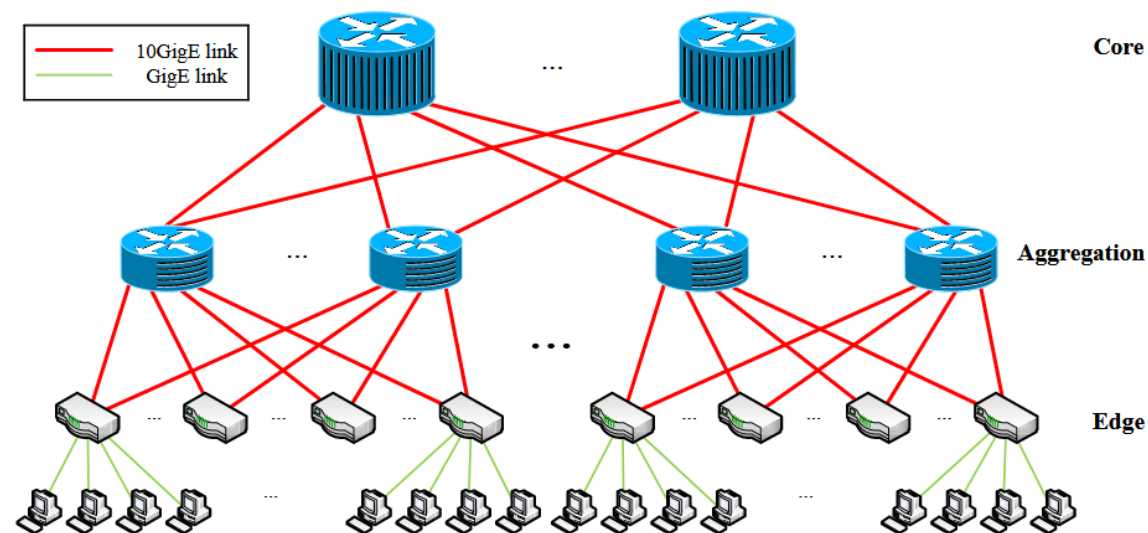
VII. Review

分布式计算的发展

- 分布式计算框架与网络服务
 - MapReduce、Hadoop、Dryad 推动商用 PC 集群上的并行计算
 - 搜索、电商与社交网络等服务，需要跨机器协同处理数据
- 数据规模带来的挑战
 - 数据集达到 PB 级，存储和计算分散在数万台机器上
 - 数据交换随规模增长，使集群内部带宽成为关键资源

网络性能瓶颈

- 现实情况：部分应用受**网络带宽**限制；
提升计算能力未必能消除瓶颈
- 拓扑演进：传统树依赖高端核心；
多根树用并行路径实现水平扩展
- 右图：路径增多后，仍需合理分配流量



Background: 2.1 Data Center Traffic Patterns

流量数据的可获得性

- 论文写作时，作者缺少可公开获取的商业数据中心流量追踪数据
- 因此，评估结合文献中的流量特征与人工构造的通信模式

两类互补的测试负载

- 参考已发表研究，生成具有局部性、随机性等特征的工作负载
- 构造跨步、热点等模式，检验网络在强竞争条件下的表现

Background: 2.2 Current Data Center Multipathing

ECMP: 按流哈希利用等价路径

- 交换机为目的子网配置多个等价下一跳，再根据报头字段哈希选路
- 同一流通常保持相同路径，减少跨路径转发导致的报文乱序
- 乱序可能触发重复确认或不必要的重传；不能等同于“乱序必然拥塞”

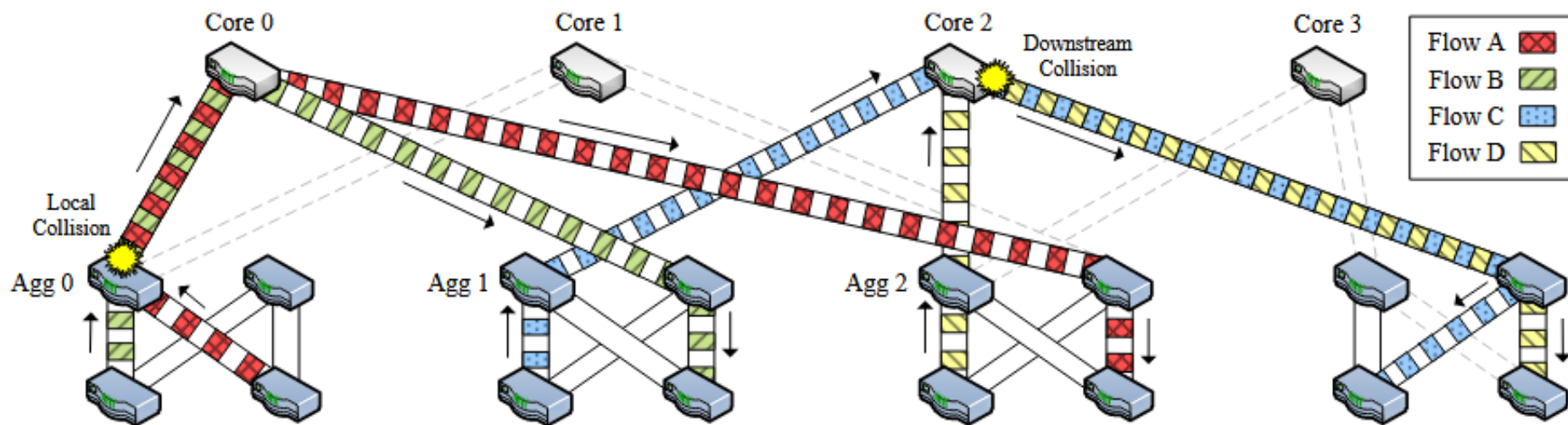
VLB: 经随机选择的中间节点转发

- 按流随机选择中间核心，分散不同连接的流量
- 论文讨论的每流 VLB 与 ECMP 具有相近的随机分流及碰撞问题

Background: 2.2 Current Data Center Multipathing

ECMP 的核心局限：大流哈希碰撞

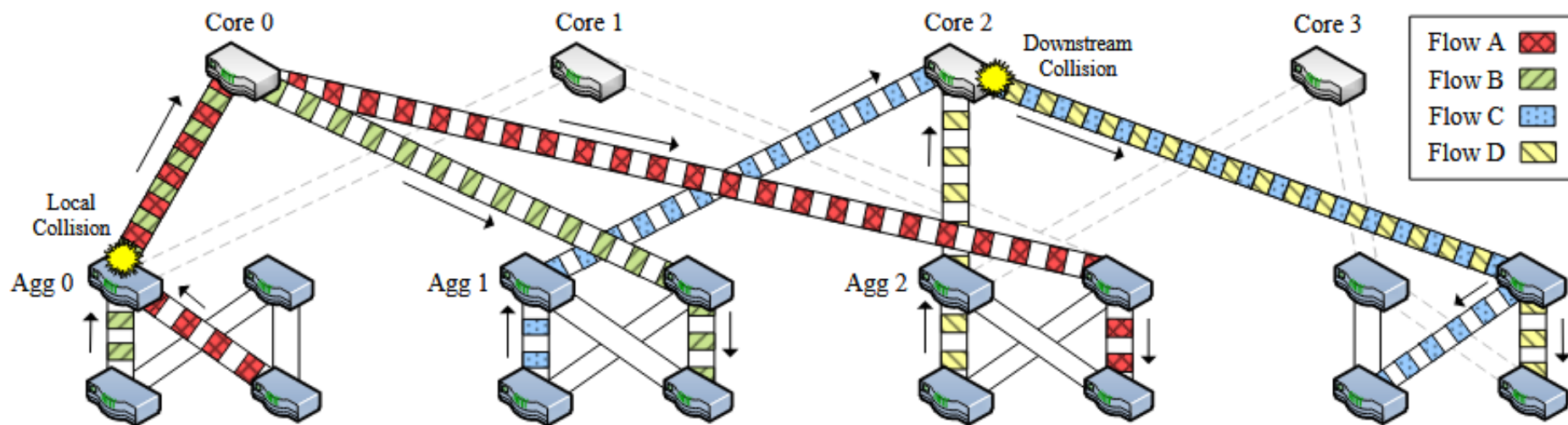
- 两个或多个持续大流可能映射到同一出口，竞争有限容量，而其他路径仍有空闲（见下图）



Background: 2.2 Current Data Center Multipathing

同一张图中的两类碰撞

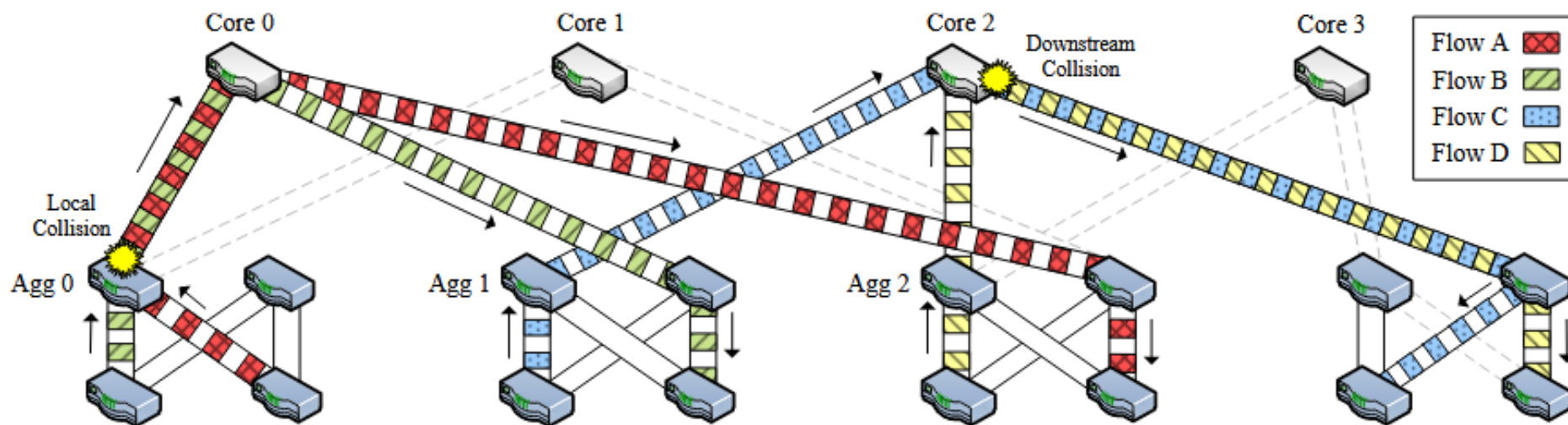
- 下图假设链路容量均为 1 Gbps
 - **本地碰撞**: A 、 B 同经 $\text{Agg0} \rightarrow \text{Core0}$, 各只能获得约 500 Mbps
 - **下游碰撞**: Agg1 、 Agg2 独立选路, C 、 D 却在 Core2 下游竞争带宽



Background: 2.2 Current Data Center Multipathing

改路能回收原本闲置的容量

- 在下图中，将冲突流分散到其他核心即可改善吞吐：
 - 流 A 改经 Core1；流 D 改经 Core3
 - 四条流均有机会达到 1 Gbps；原来的 50% 损失来自可避免的路径竞争



Background: 2.2 Current Data Center Multipathing

ECMP 的效果取决于流量结构

- 关键因素：**流的带宽需求、持续时间及每主机并发流数**
 - 少量持续大流：一次碰撞就可能占据大量链路容量
- 较适合哈希转发的场景
 - 大量同时进行的 All-to-All 流量：单条流份额通常更小
 - 仅持续几个 RTT 的短流：集中调度可能来不及产生收益
- 持续性的大流传输更值得精细调度，具体收益将在评估部分检验

Background: 2.2 Current Data Center Multipathing

蒙特卡洛模拟：量化哈希损失

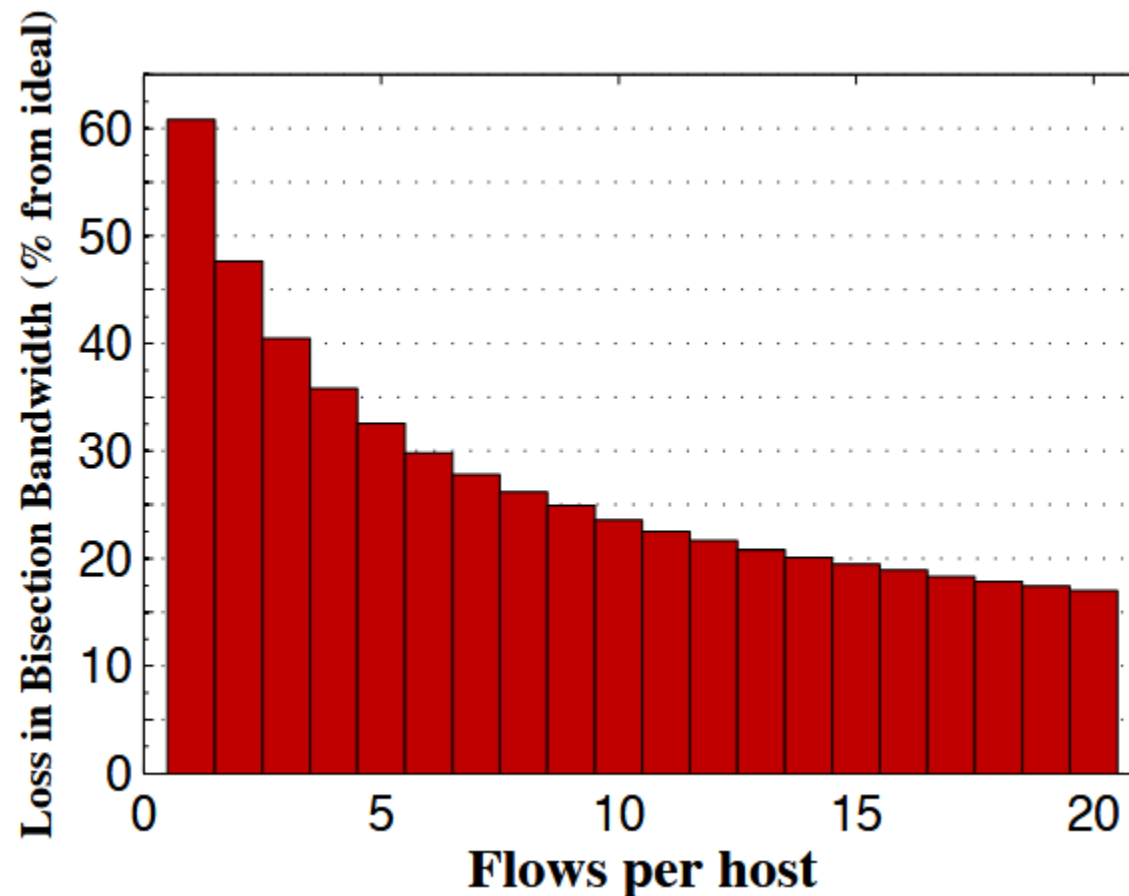
- 作者用三层 Fat-Tree 模拟数据洗牌，比较不同并发流数下的带宽损失
- 模拟配置
 - 设备：48 端口、1 GigE 交换机；各链路容量均为 1 Gbps
 - 规模：约 2.7 万台主机；流按哈希结果分配路径
 - 课堂回顾：标准 k 端口胖树的主机数为 $k^3/4$
 - 当 $k=48$ 时，可连接 27,648 台主机

Background: 2.2 Current Data Center Multipathing

少量大流时的带宽损失

➤ 顺序传输场景

- 每台主机依次向远端主机传输等量数据
- 哈希碰撞导致平均二分带宽损失 **60.8%**
- 右图：并发流较少时，单次碰撞代价更大



Background: 2.2 Current Data Center Multipathing

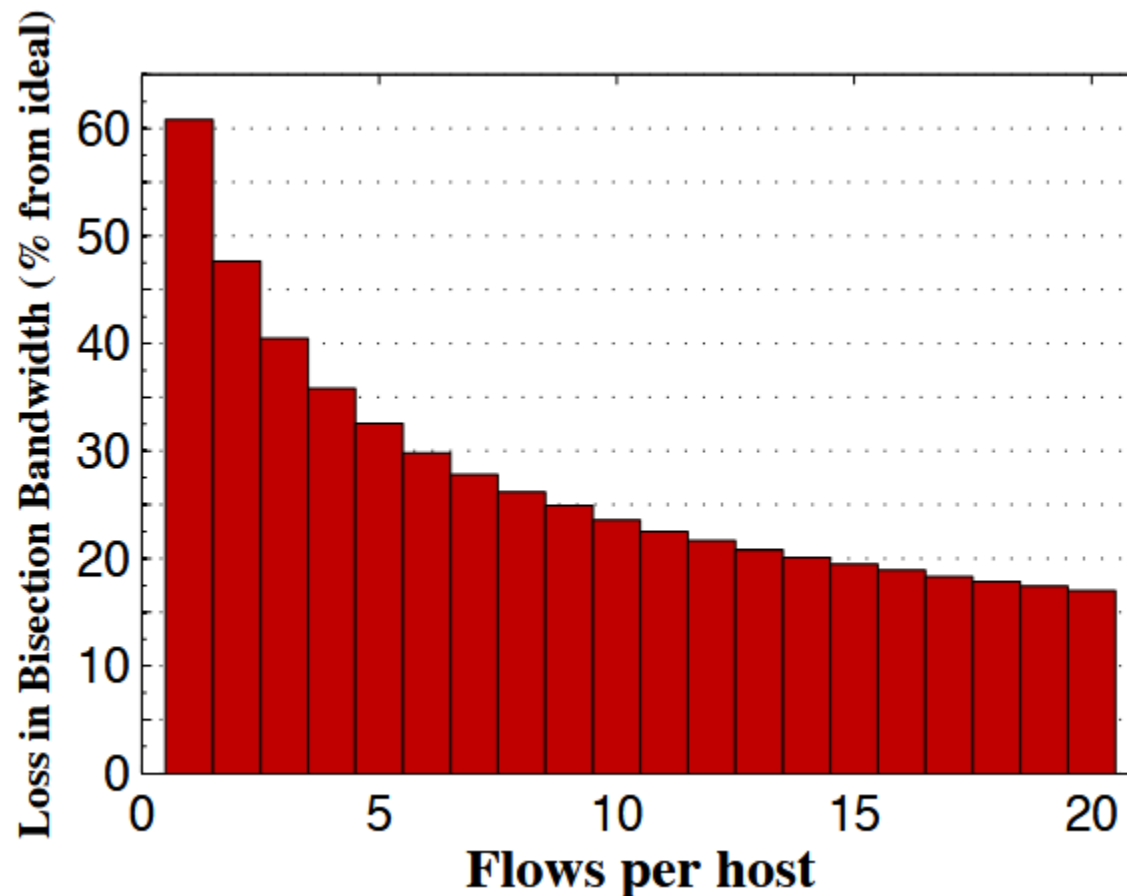
并发增多后，碰撞代价减小

➤ 并行传输场景

➤ 每主机与远端并发传输
1,000 条流

➤ 此时带宽损失为 **2.5%**

➤ 右图画出了 1—20 条流；
1,000 条流的结果见正文



Background: 2.2 Current Data Center Multipathing

为什么高并发更容易被哈希均衡?

- 决定损失的不只是碰撞次数，还包括**碰撞流占用的容量份额**
 - 固定网卡容量下的分摊
 - 并发流越多，单条流通常分到的发送速率越小
 - 少数低速流碰到一起，往往比几条满速大流碰撞的代价更低
 - “1,000 个槽位” 是直观类比，并不是交换机的硬性阈值
- 因此，大流更可能受益于调度；大量并发小流的收益可能较小

Background: 2.2 Current Data Center Multipathing

Hedera 的定位：补充默认 ECMP

- 保留 ECMP 的低开销转发，对值得优化的大流动态选路
 - 默认行为：新流先通过哈希选择等价路径
 - 调度对象：检测持续占用较多带宽、可能造成冲突的流
 - 性能收益：减少可避免的竞争，回收闲置路径容量
 - 部署思路：沿用现有主机网络栈，通过交换机规则执行调整
- 下一步：改路之前，需要知道每条流真正希望获得多少带宽

Background: 2.3 Dynamic Flow Demand Estimation

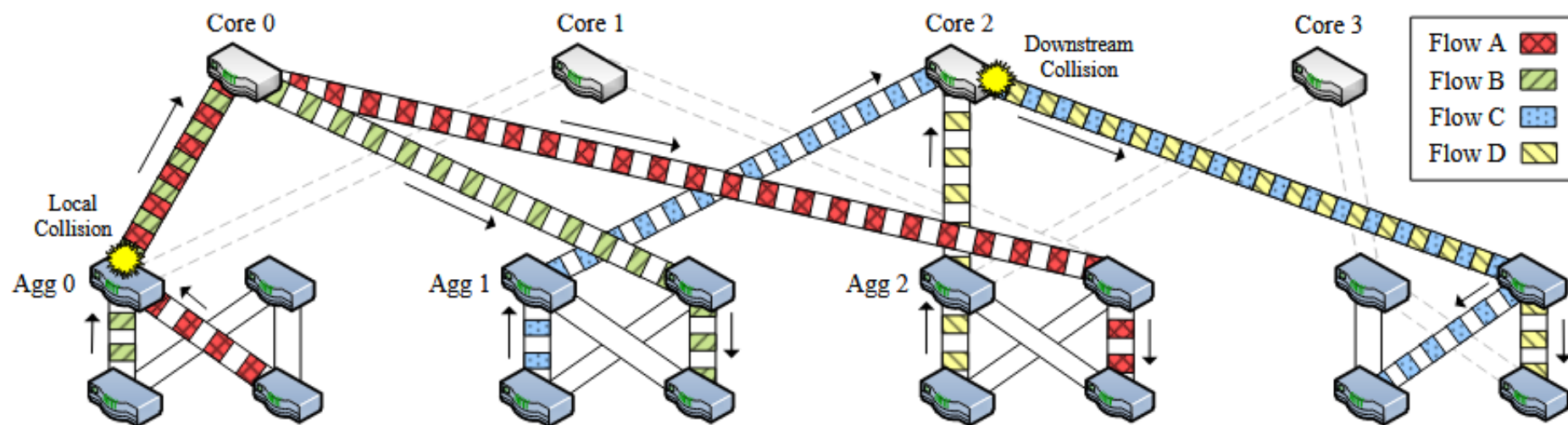
动态调度先要回答：该搬哪些流？

- 直观做法：监测链路利用率，把流从拥塞路径移到空闲路径
 - 测量当前流速，可以知道流现在消耗了多少带宽
- 关键难点：**当前速率可能已经被旧路径上的拥塞压低**
 - 若按这一低速率预留新路径，改路后 TCP 增速可能再次造成拥塞
- 因此需要估计流在更理想条件下的**自然需求**，再决定路径放置

Background: 2.3 Dynamic Flow Demand Estimation

当前速率不等于自然需求

- 下图中，每条流当前约为 500 Mbps；合理改路后可达到 1 Gbps
- 观测到的低速率可能是路径拥塞的结果，不能直接视为流的需求上限



Background: 2.3 Dynamic Flow Demand Estimation

自然需求：解除内部瓶颈后的带宽份额

- TCP 流的**自然需求** (natural demand)
 - 在理想无阻塞网络中，仅受发送端与接收端容量限制时可获得的速率
 - 环境假设：忽略内部链路瓶颈，保留端点网卡容量约束
 - 共享关系：多个流仍须分享同一源或目的的容量，并非每流都能满速
 - 调度价值：避免用受拥塞压低的速率，为新路径低估容量需求
- 第 4.2 节用最大最小公平分配估计这些份额

Outline

- I. Introduction
- II. Background
- III. Architecture**
- IV. Estimation and Scheduling
- V. Implementation
- VI. Evaluation
- VII. Review

Architecture: 3.0 Overview

Hedera 的三步控制闭环

- 大流检测：边缘交换机提供统计信息，识别高带宽流
- 需求估计与路径计算：估计自然需求，搜索更合理的路径安排
- 路径部署：向交换机安装规则，再观察流量变化
- 系统定位
 - 面向多根树网络；论文在 Fat-Tree 测试床上实现并验证
 - 控制器负责全局优化，数据包仍在交换机中按规则转发

Architecture: 3.1 Switch Initialization

上行选路，下行确定转发

- 利用路径多样性
 - 上行阶段可以选择不同聚合与核心交换机，分散源主机的出向流量
- 保持可执行的路径结构
 - 上行：为一条流选择合适路径；不是让同一流的每个包随机跳路
 - 下行：选定核心后，按目的地址前缀沿确定的最小代价路径到达主机

Architecture: 3.1 Switch Initialization

交换机初始化与新流处理

- 核心匹配目的 pod 前缀；聚合匹配目的边缘前缀；边缘直接交付主机
- 默认路径：以流的 10 元组做哈希，选择等价路径
- 调度门槛：原型为 **100 Mbps**，即主机 1 GigE 链路容量的 10%
- 流定义：实验中将具有相同 10 元组的数据包归入同一流

<src MAC, dst MAC, src IP, dst IP, EtherType, IP protocol,
TCP src port, dst port, VLAN tag, input port>

Architecture: 3.2 Scheduler Design

中心调度器：全局观察，动态优化

- 目标：避免让同一链路上的合计自然需求超过容量，提升聚合二分带宽
- 触发：流持续存在且达到检测门槛后，进入路径放置过程
- 安装：修改源 pod 的边缘与聚合交换机规则，引导流量到所选路径
- 状态：规则随流结束而超时；调度状态主要用于性能优化
- 算法：比较 Global First Fit、Simulated Annealing 与静态 ECMP
- 容错：依赖底层 PortLand 机制获知链路 / 交换机故障，再重选可用路径

Outline

I. Introduction

II. Background

III. Architecture

IV. Estimation and Scheduling

V. Implementation

VI. Evaluation

VII. Review

Estimation and Scheduling: 4.1 Flow Placement

从需求估计走向路径放置

- 多商品流问题：为共享网络中的多组源—目的流量选择路径，并满足链路容量约束
 - 一般网络中的整数流版本属于 NP 完全问题；不同拓扑的复杂度并不相同
- Hedera 采用**实用启发式算法**，在求解时间与可获得的带宽之间取舍
 - 在讨论放置之前，先区分瓶颈来自网络还是主机

Estimation and Scheduling: 4.1 Flow Classification

两类流：网络受限与主机受限

➤ 网络受限流

- 有充足数据可发送，但路径拥塞压低速率；更合理的选路可能改善吞吐

➤ 主机受限流

- 吞吐受磁盘、CPU 或应用处理能力限制；增加网络带宽未必有效
- 论文重点：估计并调度网络受限的大流，主机内部瓶颈不在其解决范围内

Estimation and Scheduling: 4.2 Demand Estimation

需求估计：在端点容量之间迭代分配

- 输入活跃大流的源、目的及流数，估计理想无阻塞网络中的公平带宽份额
 - 发送端增加：扣除已收敛份额，把剩余容量均分给未收敛流
 - 接收端削减：保留较小需求，让受限流公平分享剩余接收容量
 - 交替迭代，直到需求稳定；稀疏矩阵与并行处理降低实现成本

Estimation and Scheduling: 4.2 Demand Estimation

ESTIMATE-DEMANDS()

```

1  for all  $i, j$ 
2       $M_{i,j} \leftarrow 0$ 
3  do
4      foreach  $h \in H$  do EST-SRC( $h$ )
5      foreach  $h \in H$  do EST-DST( $h$ )
6  while some  $M_{i,j}$ .demand changed
7  return  $M$ 

```

EST-SRC(src: host)

```

1   $d_F \leftarrow 0$ 
2   $n_U \leftarrow 0$ 
3  foreach  $f \in \langle \text{src} \rightarrow \text{dst} \rangle$  do
4      if  $f$ .converged then
5           $d_F \leftarrow d_F + f$ .demand
6      else
7           $n_U \leftarrow n_U + 1$ 
8   $e_S \leftarrow \frac{1.0 - d_F}{n_U}$ 
9  foreach  $f \in \langle \text{src} \rightarrow \text{dst} \rangle$  and not  $f$ .converged do
10      $M_{f.\text{src}, f.\text{dst}}.\text{demand} \leftarrow e_S$ 

```

EST-DST(dst: host)

```

1   $d_T, d_S, n_R \leftarrow 0$ 
2  foreach  $f \in \langle \text{src} \rightarrow \text{dst} \rangle$ 
3       $f.\text{rl} \leftarrow \text{true}$ 
4       $d_T \leftarrow d_T + f$ .demand
5       $n_R \leftarrow n_R + 1$ 
6  if  $d_T \leq 1.0$  then
7      return
8   $e_S \leftarrow \frac{1.0}{n_R}$ 
9  do
10      $n_R \leftarrow 0$ 
11     foreach  $f \in \langle \text{src} \rightarrow \text{dst} \rangle$  and  $f.\text{rl}$  do
12         if  $f$ .demand  $< e_S$  then
13              $d_S \leftarrow d_S + f$ .demand
14              $f.\text{rl} \leftarrow \text{false}$ 
15         else
16              $n_R \leftarrow n_R + 1$ 
17      $e_S \leftarrow \frac{1.0 - d_S}{n_R}$ 
18     while some  $f.\text{rl}$  was set to false
19     foreach  $f \in \langle \text{src} \rightarrow \text{dst} \rangle$  and  $f.\text{rl}$  do
20          $M_{f.\text{src}, f.\text{dst}}.\text{demand} \leftarrow e_S$ 
21          $M_{f.\text{src}, f.\text{dst}}.\text{converged} \leftarrow \text{true}$ 

```

Estimation and Scheduling: 4.2 Demand Estimation

四主机需求估计示例（下图）

- 矩阵的行是源、列是目的；分数为每流份额，下标为对应流数
- 圆括号尚未收敛，灰色方括号已收敛；箭头表示交替估计过程

$$\begin{bmatrix} & (\frac{1}{3})_1 & (\frac{1}{3})_1 & (\frac{1}{3})_1 \\ (\frac{1}{3})_2 & & (\frac{1}{3})_1 & 0_0 \\ (\frac{1}{2})_1 & 0_0 & & (\frac{1}{2})_1 \\ 0_0 & (\frac{1}{2})_2 & 0_0 & \end{bmatrix} \Rightarrow \begin{bmatrix} & [\frac{1}{3}]_1 & (\frac{1}{3})_1 & (\frac{1}{3})_1 \\ [\frac{1}{3}]_2 & & (\frac{1}{3})_1 & 0_0 \\ [\frac{1}{3}]_1 & 0_0 & & (\frac{1}{2})_1 \\ 0_0 & [\frac{1}{3}]_2 & 0_0 & \end{bmatrix} \Rightarrow \begin{bmatrix} & [\frac{1}{3}]_1 & (\frac{1}{3})_1 & (\frac{1}{3})_1 \\ [\frac{1}{3}]_2 & & (\frac{1}{3})_1 & 0_0 \\ [\frac{1}{3}]_1 & 0_0 & & (\frac{2}{3})_1 \\ 0_0 & [\frac{1}{3}]_2 & 0_0 & \end{bmatrix} \Rightarrow \begin{bmatrix} & [\frac{1}{3}]_1 & (\frac{1}{3})_1 & [\frac{1}{3}]_1 \\ [\frac{1}{3}]_2 & & (\frac{1}{3})_1 & 0_0 \\ [\frac{1}{3}]_1 & 0_0 & & [\frac{2}{3}]_1 \\ 0_0 & [\frac{1}{3}]_2 & 0_0 & \end{bmatrix}$$

- 先由发送端分配，再按接收端容量削减；释放的发送容量重新分配
- 端点总负载 = 每流份额 × 流数，不能直接把矩阵数值相加

Estimation and Scheduling: 4.2 Demand Estimation

```
EST-SRC(src: host)
1   $d_F \leftarrow 0$ 
2   $n_U \leftarrow 0$ 
3  foreach  $f \in \langle \text{src} \rightarrow \text{dst} \rangle$  do
4    if  $f.\text{converged}$  then
5       $d_F \leftarrow d_F + f.\text{demand}$ 
6    else
7       $n_U \leftarrow n_U + 1$ 
8   $e_S \leftarrow \frac{1.0 - d_F}{n_U}$ 
9  foreach  $f \in \langle \text{src} \rightarrow \text{dst} \rangle$  and not  $f.\text{converged}$  do
10    $M_{f.\text{src}, f.\text{dst}}.\text{demand} \leftarrow e_S$ 
```

发送方“灌水”：分配剩余容量

- 目标：计算从 src 发出的各流份额
- 公式： $e_S = (1 - d_F) / n_U$
 d_F 为已收敛需求总和, n_U 为未收敛流数
- 行 9—10：把 e_S 分配给未收敛流；已收敛份额保持不变
- 算例：已收敛份额为 0.4，剩余两条流各得到 $(1 - 0.4) / 2 = 0.3$

Estimation and Scheduling: 4.2 Demand Estimation

```

EST-DST(dst: host)
1   $d_T, d_S, n_R \leftarrow 0$ 
2  foreach  $f \in \langle \text{src} \rightarrow \text{dst} \rangle$ 
3       $f.\text{rl} \leftarrow \text{true}$ 
4       $d_T \leftarrow d_T + f.\text{demand}$ 
5       $n_R \leftarrow n_R + 1$ 
6  if  $d_T \leq 1.0$  then
7      return
8   $e_S \leftarrow \frac{1.0}{n_R}$ 
9  do
10      $n_R \leftarrow 0$ 
11     foreach  $f \in \langle \text{src} \rightarrow \text{dst} \rangle$  and  $f.\text{rl}$  do
12         if  $f.\text{demand} < e_S$  then
13              $d_S \leftarrow d_S + f.\text{demand}$ 
14              $f.\text{rl} \leftarrow \text{false}$ 
15         else
16              $n_R \leftarrow n_R + 1$ 
17          $e_S \leftarrow \frac{1.0 - d_S}{n_R}$ 
18     while some  $f.\text{rl}$  was set to false
19     foreach  $f \in \langle \text{src} \rightarrow \text{dst} \rangle$  and  $f.\text{rl}$  do
20          $M_{f.\text{src}, f.\text{dst}}.\text{demand} \leftarrow e_S$ 
21          $M_{f.\text{src}, f.\text{dst}}.\text{converged} \leftarrow \text{true}$ 
    
```

接收方“校准”：避免总需求超过 1

- 目标：入向需求总和不超过归一化的接收容量
- 行 6—7：若总需求 $d_T \leq 1$ ，直接返回
- 行 8—18：保留较小需求，重算其他流的公平份额
- 行 19—21：更新受接收端限制的流，并标记收敛
- 算例：0.2、0.6、0.6
调整为 0.2、0.4、0.4

Estimation and Scheduling: 4.3 Global First Fit

全局首次适应 (Global First Fit)

- 利用估计需求，依次检查候选路径
- 找到各链路都装得下的第一条路径，就记录容量预留并返回该路径
- 右上图：一次贪心放置的过程
 - 行 1—2：保留已有分配；行 3—6：检查路径并更新容量记录
 - 返回路径后，系统再安装转发规则；第 6 行本身不是安装流表
 - 若无路径可完整容纳需求，则回退到哈希选路；流结束时释放预留
 - 特点：简单、可增量处理；已有选择会限制后续放置，不保证全局最优

```
GLOBAL-FIRST-FIT( $f$ : flow)
1   if  $f$ .assigned then
2       return old path assignment for  $f$ 
3   foreach  $p \in P_{\text{src} \rightarrow \text{dst}}$  do
4       if  $p.\text{used} + f.\text{rate} < p.\text{capacity}$  then
5            $p.\text{used} \leftarrow p.\text{used} + f.\text{rate}$ 
6           return  $p$ 
7   else
8        $h = \text{HASH}(f)$ 
9       return  $p = P_{\text{src} \rightarrow \text{dst}}(h)$ 
```

Estimation and Scheduling: 4.4 Simulated Annealing

为什么还需要模拟退火？

- 全局首次适应开销低，但通常保留已有安排，缺少主动重排布局的能力
 - 网络负载升高后，可用容量变得零散，贪心选择更容易受到限制
 - 模拟退火通过概率搜索调整全局布局，增加离开局部极小值的机会
- 有限次搜索仍**不保证全局最优**；实际收益需要实验检验
- Hedera 的关键在于：如何表示状态、评价状态并快速生成邻居

Estimation and Scheduling: 4.4 Simulated Annealing

模拟退火：利用拓扑结构缩小搜索空间

- 直接做法：为每条流独立选核心，组合空间随流数迅速增长
- 本文做法：为目的主机分配核心；需要经过核心层的流据此选择路径
- 在每个 pod 内建立主机与核心的一一映射，通过交换两台主机的核心产生邻居
- 能量：累加所有链路超出容量的需求；只更新受交换影响的相关链路
- 跨周期复用上轮最佳状态，减少扰动；该受限搜索空间有其适用条件

Estimation and Scheduling: 4.4 Simulated Annealing

模拟退火算法 Pseudocode (右图)

- 当前状态与历史最佳状态分别保存
 - 初始化: 设置 s 、能量 $E(s)$ 和温度 T
 - 生成邻居, 计算新能量
 - 若刷新最佳值, 更新历史最佳状态
 - 按能量差与温度决定是否接受邻居
 - 逐步降温, 直到完成给定迭代次数
 - 返回历史最佳状态; 不保证全局最优

```
SIMULATED-ANNEALING( $n$  : iteration count)
1    $s \leftarrow \text{INIT-STATE}()$ 
2    $e \leftarrow E(s)$ 
3    $s_B \leftarrow s, e_B \leftarrow e$ 
4    $T_0 \leftarrow n$ 
5   for  $T \leftarrow T_0 \dots 0$  do
6        $s_N \leftarrow \text{NEIGHBOR}(s)$ 
7        $e_N \leftarrow E(s_N)$ 
8       if  $e_N < e_B$  then
9            $s_B \leftarrow s_N, e_B \leftarrow e_N$ 
10      if  $P(e, e_N, T) > \text{RAND}()$  then
11           $s \leftarrow s_N, e \leftarrow e_N$ 
12  return  $s_B$ 
```

Estimation and Scheduling: 4.5 Algorithm Comparison

模拟退火算法 vs. 全局首次适应

- 响应方式: GFF 可在检测后放置; SA 按周期搜索, 并复用上轮布局
- 解的质量: SA 通常利用率更高, 但两种算法都不保证全局最优
- 下表为单次放置 / 邻居操作的复杂度; 总成本还受流数和迭代次数影响
- 工程取舍: 同时考虑计算时间、响应速度与带宽收益

Algorithm Complexity	Time	Space
Global First-Fit	$O((k/2)^2)$	$O(k^3 + F)$
Simulated Annealing	$O(f_{avg})$	$O(k^3 + F)$

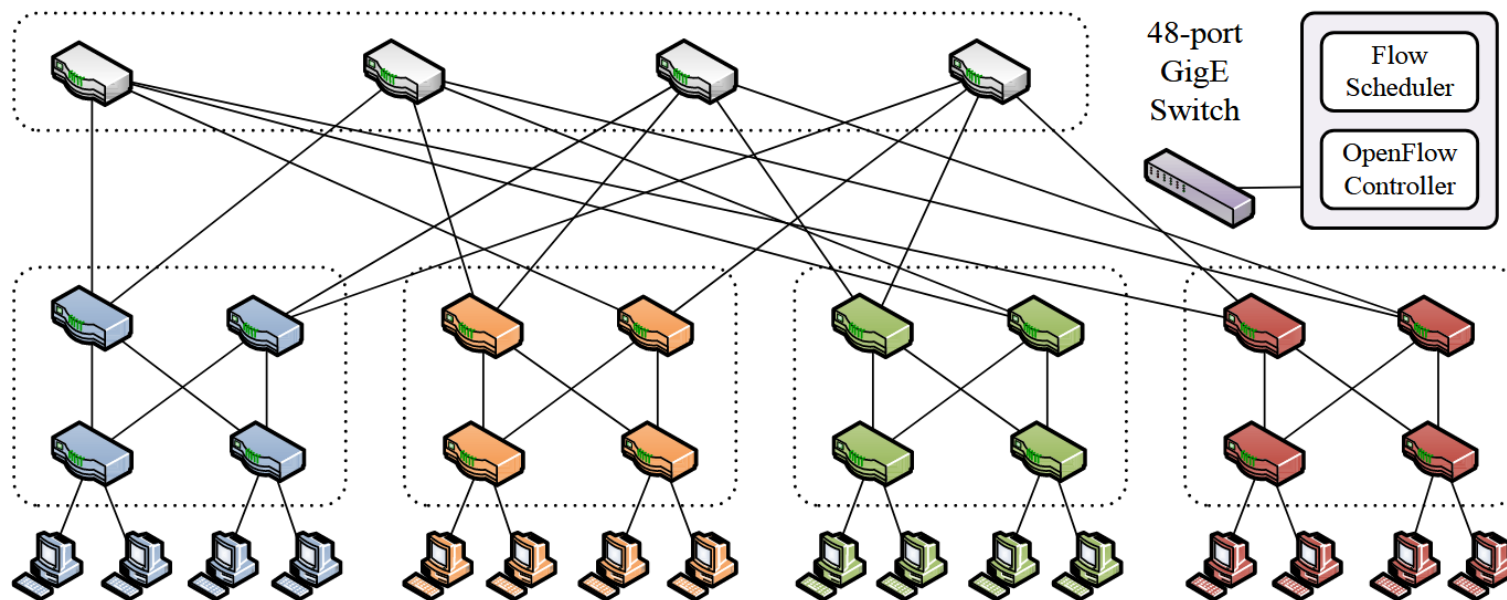
Outline

- I. Introduction
- II. Background
- III. Architecture
- IV. Estimation and Scheduling
- V. Implementation**
- VI. Evaluation
- VII. Review

Implementation: 5.1 Topology

真实测试床与大规模仿真相互补充

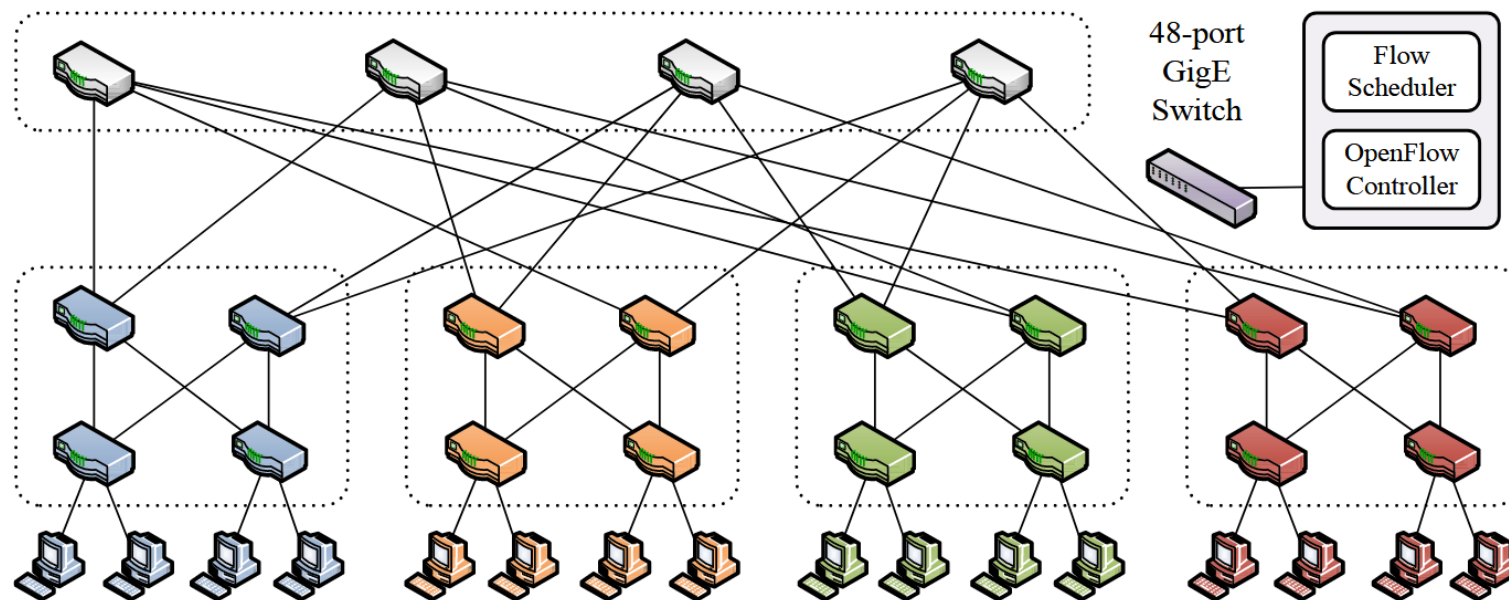
- 物理测试床：检验监测、选路和流表更新能否在真实设备上工作
- 流级仿真：扩大主机与流的数量，评估动态调度的收益和扩展性
- Fat-Tree 测试床（右图）
 - 16 台主机； $k = 4$
 - 20 台四端口交换机；数据链路为 1 GigE
 - 独立 48 端口交换机承载控制消息



Implementation: 5.1 Topology

测试床控制平面

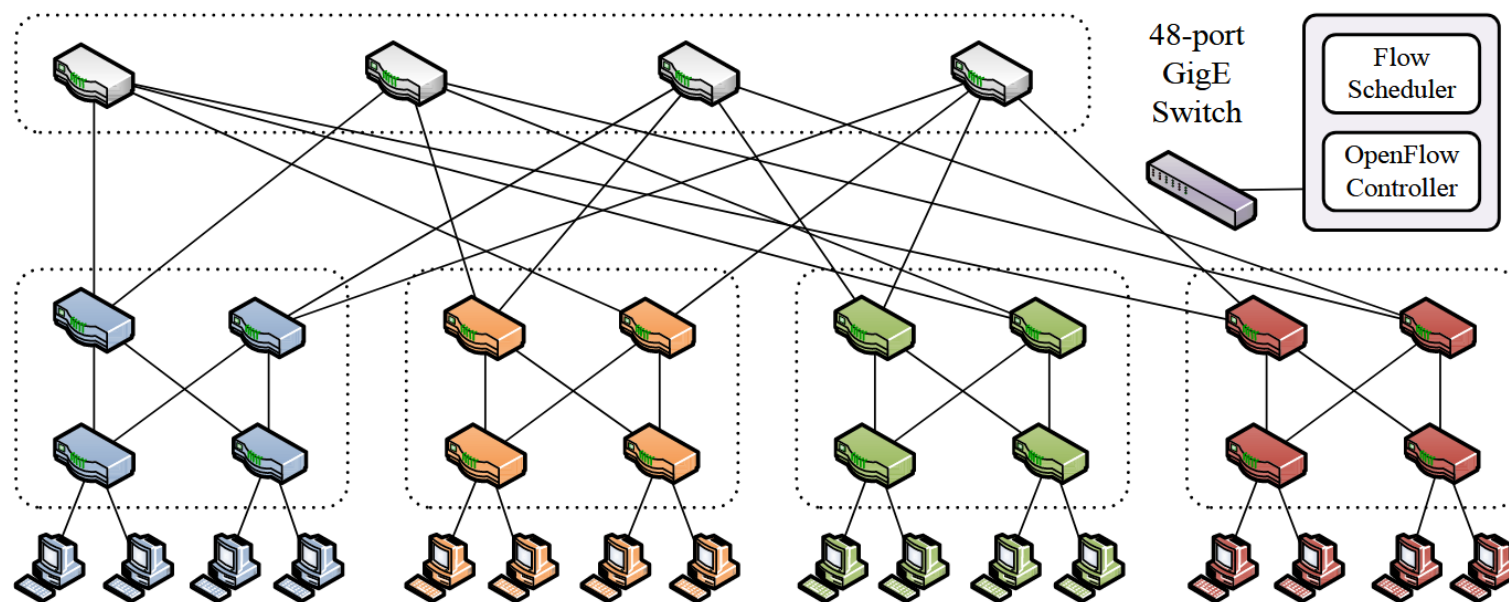
- 独立控制网络传输统计、配置与路径更新消息
- 原型以此方便测试和调试；控制消息也可通过数据平面传输
- 扩展：控制网络可采用传统树形结构
- 调度器在独立机器上运行
- 原型将控制与数据分离
- 主机网络栈无需修改



Implementation: 5.1 Topology

回顾：Fat-Tree 的容量与路径多样性

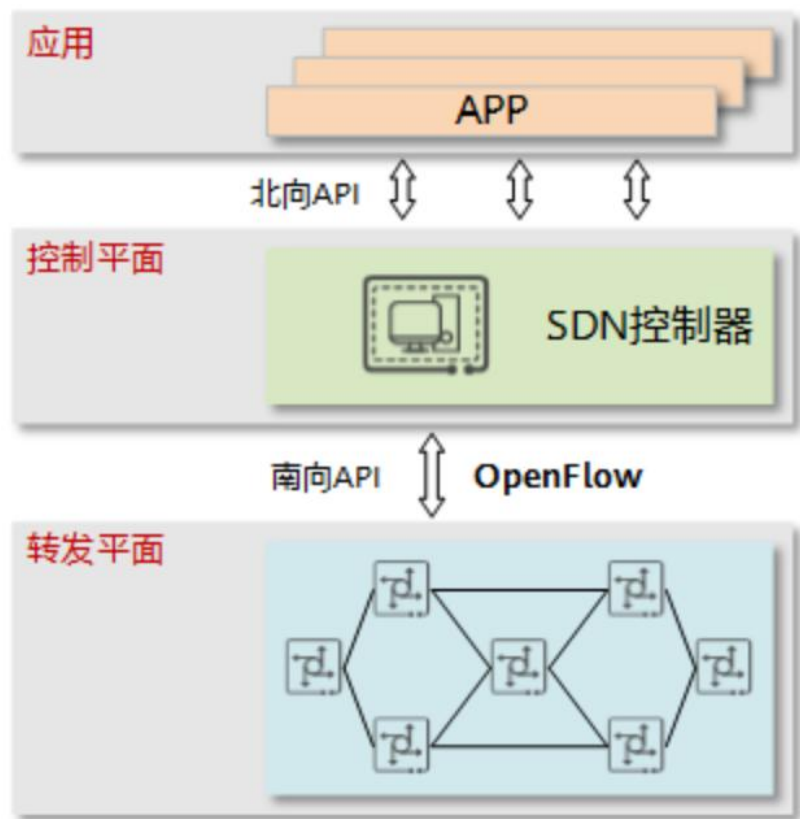
- k 个 pod; 每个 pod 含 $k/2$ 台边缘和 $k/2$ 台聚合交换机
- 每台边缘连接 $k/2$ 台主机; 总主机数为 $k^3/4$
- 核心交换机数量为 $(k/2)^2$
- 不同 pod 的主机对有 $(k/2)^2$ 条等价核心路径
- 这些路径并非完全链路不相交
- 仍需调度, 避免多条流竞争相同链路



Implementation: 5.3 OpenFlow Control

OpenFlow: 访问交换机转发表

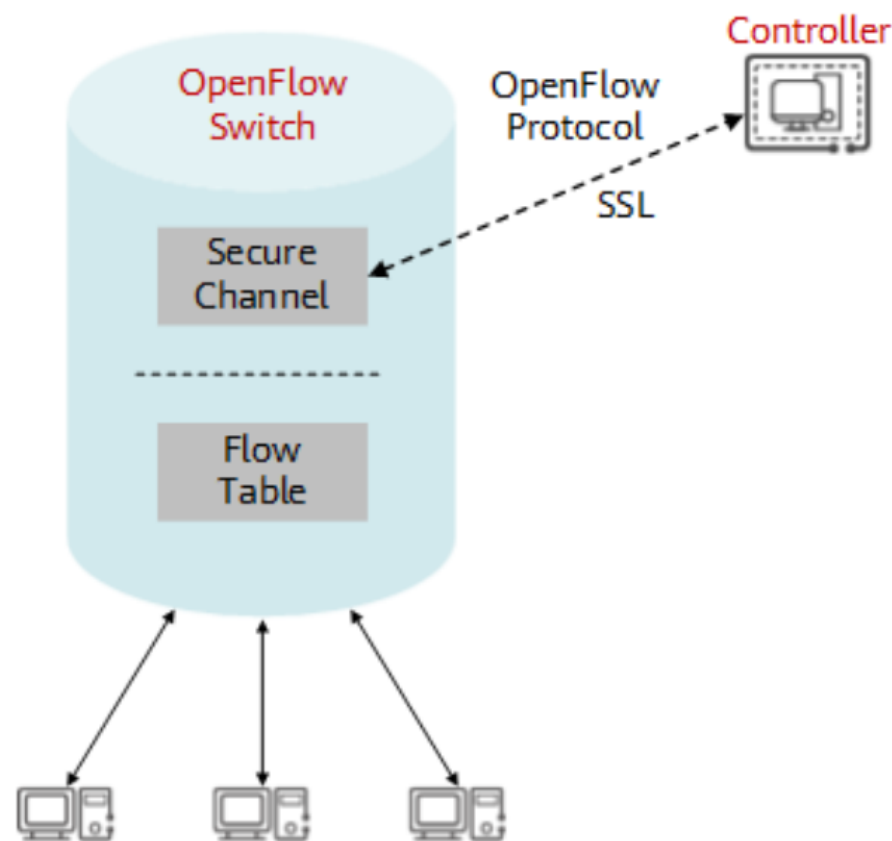
- Hedera 通过 OpenFlow 查询统计和更新规则；当时已有多种交换机实现
- 交换机按流表匹配报文，并执行指定动作
 - 转发：选择一个或多个输出端口
 - 丢弃：过滤不应继续转发的报文
 - 复制：向多个端口发送，用于广播或多播



Implementation: 5.3 OpenFlow Control

控制器接口与交换机统计

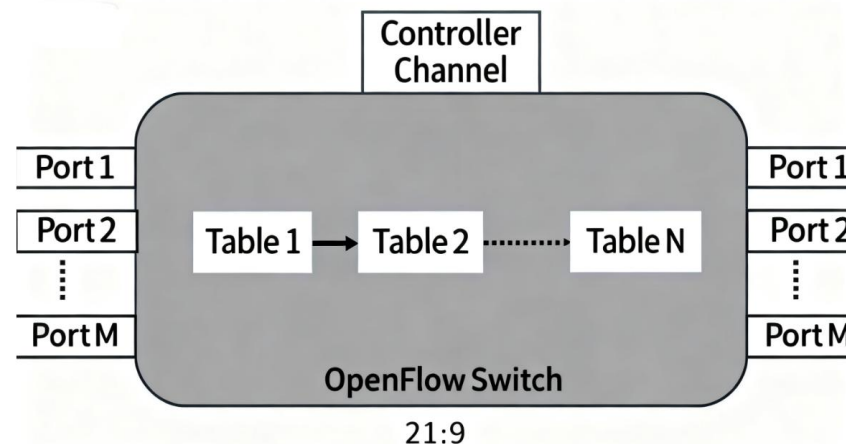
- 交换机与控制器建立控制通道，用于查询和修改流表
 - 查询：获取流状态及相关计数器
 - 操作：插入、修改或删除流表项
- 用于检测大流的统计
 - 流级：字节数、持续时间等
 - 端口级：各端口的流量与负载情况



Implementation: OpenFlow and Scheduling

默认转发与硬件执行

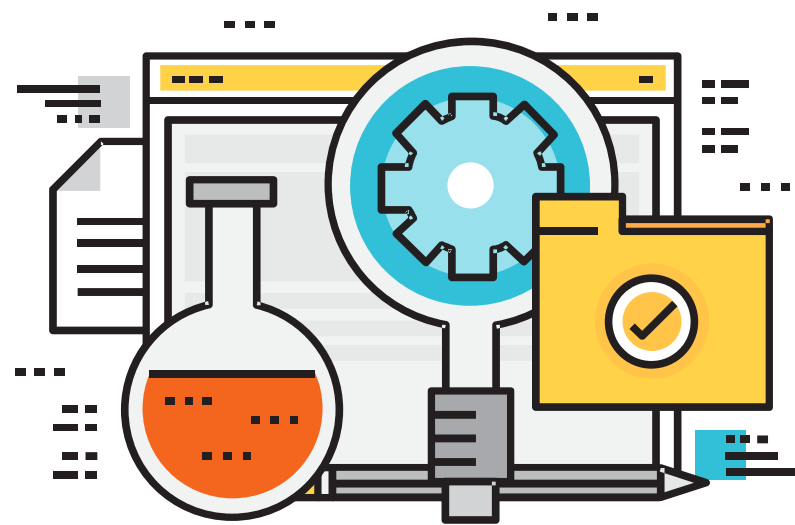
- 新流先建立默认 ECMP 规则
 - 根据流哈希选择等价出口
 - 后续报文按规则由硬件转发
- 检测大流 → 估计需求与选路 → 更新转发规则
 - 原型调度周期：每 5 秒轮询一次，受 NetFPGA 寄存器读取速率限制
 - 计算结果用于改善路径放置，不承诺每次都找到最优解
- 后文的约百毫秒控制开销是带假设的估算，不能当作原型已实现的周期



Implementation: 5.5 Simulator

流级仿真：扩大规模，保留模型边界

- 从 16 主机测试床扩展到最多 **8,192 台主机**的动态流量场景
- 逐包模拟规模过大，因此按流更新速率、到达与结束事件
 - 模型近似 TCP 慢启动和 AIMD，不逐包重现缓冲区与重传超时
 - 因此争用场景的吞吐可能偏乐观，不能直接当作真实网络的保证值



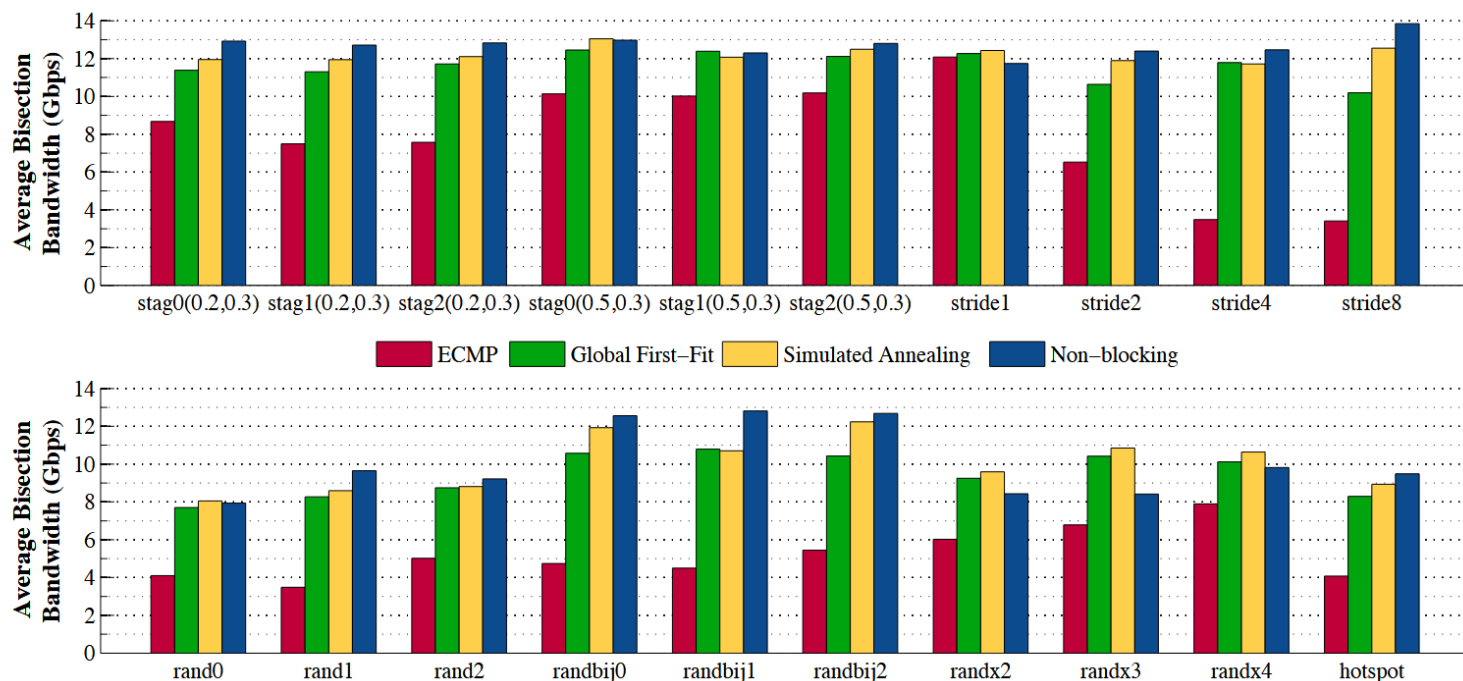
Outline

- I. Introduction
- II. Background
- III. Architecture
- IV. Estimation and Scheduling
- V. Implementation
- VI. Evaluation**
- VII. Review

Evaluation: 6.2 Testbed Benchmark Results

真实测试床：多种通信模式下的吞吐

- 下图：在多数受测模式中，GFF 和 SA 明显优于静态 ECMP
- 收益随通信局部性和碰撞程度变化；不同负载应与各自参照比较



- 左图：通信模式对比
上：Staggered / Stride
下：随机、双射、热点
Non-blocking：
同负载的无阻塞实测参照

Evaluation: 6.3 Data Shuffle

120 GB Shuffle: 带宽与完成时间分别比较

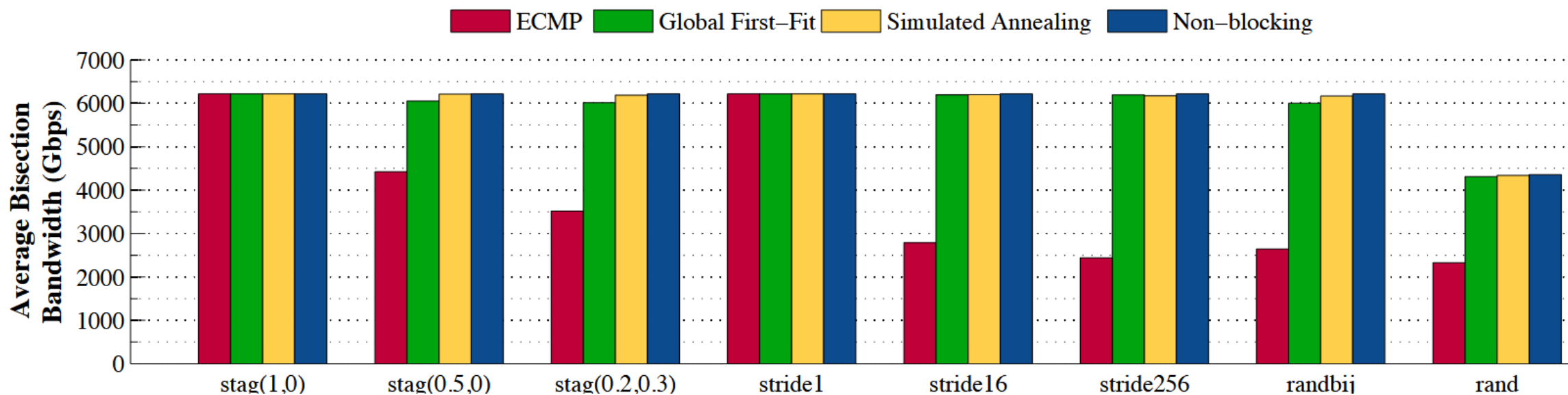
- 16 台主机分别依次向其余 15 台发送 500 MB，构成 120 GB 数据洗牌
- 下表：GFF 相比 ECMP，平均带宽从 2.811 提升到 3.891 Gbps（约 +38.4%）
 - 总完成时间从 438.44 降到 335.50 秒（约 -23.5%），不能混用两个比例
 - 该实验是顺序式大流传输；大量并发短流的调度收益可能更小

	ECMP	GFF	SA	Control
Shuffle time (s)	438.44	335.50	335.96	306.37
Host completion (s)	358.14	258.70	261.96	226.56
Bisec. BW (Gbps)	2.811	3.891	3.843	4.443
Goodput (MB/s)	20.94	28.99	28.63	33.10

Evaluation: 6.4.1 Communication Patterns

8,192 主机仿真：动态调度仍有收益

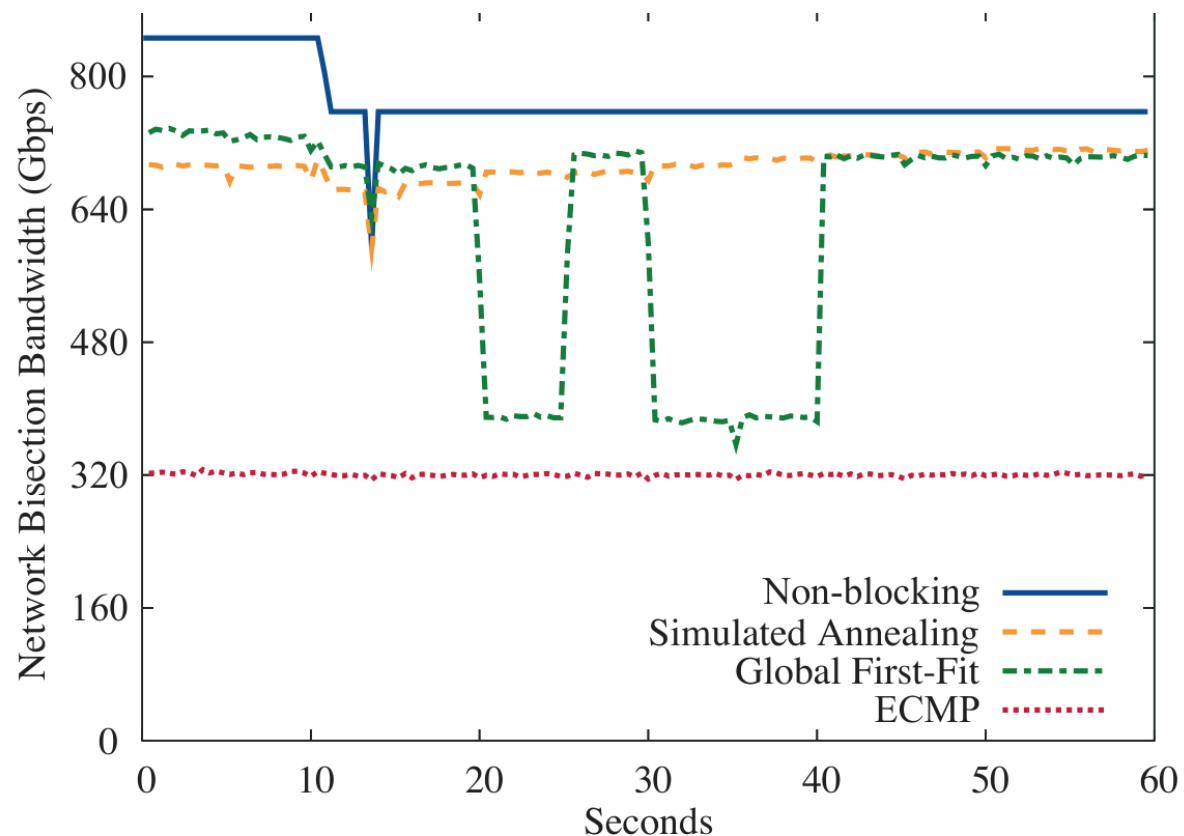
- 下图：在多数通信模式中，动态调度比静态 ECMP 获得更高带宽
- 全局放置减少可避免的路径竞争，尤其有利于跨 pod 的持续大流
- 与 Non-blocking 的差距，应结合负载、搜索算法和仿真假设分析



Evaluation: 6.4.1 Bandwidth over Time

动态过程中的聚合吞吐

- 右图：1,024 台主机，随机双射流量
 - 动态算法多数时间接近无阻塞参照
 - 曲线局部下跌，反映了平均值掩盖的波动
 - 评估指标为吞吐；尾延迟需另行评估



Evaluation: 6.4.2 Quality of Simulated Annealing

迭代收益递减，比较时注意分母

- 右表：迭代增加后，带宽改善趋缓；
结果不要求严格单调
- 百分比按满网络二分带宽归一化；
同负载无阻塞参照也低于 100%
 - 8,192 主机、100 万次迭代：75.03%
 - 同负载 Non-blocking: 77.63%
 - $75.03 / 77.63 \approx 96.65\%$
 - **评价收益时须使用同负载参照；
实际迭代预算还需兼顾计算耗时**

	Number of Hosts		
SA Iterations	16	1,024	8,192
1000	78.73	74.69	72.83
50000	78.93	75.79	74.27
100000	78.62	75.77	75.00
500000	79.35	75.87	74.94
1000000	79.04	75.78	75.03
1500000	78.71	75.82	75.13
2000000	78.17	75.87	75.05
Non-blocking	81.24	78.34	77.63

Evaluation: 6.4.3 Demand Estimation Runtime

需求估计耗时 (右表)

- 实测配置：四核 2.13 GHz 机器，采用 4 个并行线程
 - 27,648 主机、10 万大流：51.30 ms
 - 25 万大流：199.43 ms，不能概括为全部低于 100 ms
 - 稀疏矩阵与并行处理降低成本，但流数增加仍会提高开销

k	Hosts	Large flows	Runtime (ms)
16	1024	1024	1.45
16	1024	5000	4.14
32	8192	8192	2.71
32	8192	25000	9.23
32	8192	50000	26.31
48	27648	27648	6.91
48	27648	100000	51.30
48	27648	250000	199.43

Evaluation: 6.4.4 Complexity of Simulated Annealing

模拟退火计算耗时：流数与迭代数都重要

- 下表测量路径搜索耗时；不能概括为完整控制周期与网络规模无关
- 8,192 主机、50,000 流：1,000 次约 11.573 ms；10,000 次约 55.741 ms
- 结合迭代收益选择计算预算，并把需求估计、消息传输和规则安装计入闭环

	1,024 hosts		8,192 hosts	
Iterations	$f = 3,215$	$f = 6,250$	$f = 25k$	$f = 50k$
1000	2.997	5.042	6.898	11.573
5000	12.209	20.848	19.091	32.079
10000	23.447	40.255	32.912	55.741

Evaluation: 6.4.5 Control Overhead

控制闭环：上报、计算与下发

- 下表对完整控制回路作数量级分析，不是大规模实测
- 假设：计算合计 100 ms、每流消息 72 B，控制器最后一跳为 10 GigE
- 27,648 主机、每主机 10 条大流时，估算闭环为 145.5 ms；结果取决于这些条件

	Flows per host		
Hosts	1	5	10
1,024	100.2	100.9	101.7
8,192	101.4	106.8	113.5
27,648	104.6	122.8	145.5

Table 6: Length of control loop (ms).

Outline

- I. Introduction
- II. Background
- III. Architecture
- IV. Estimation and Scheduling
- V. Implementation
- VI. Evaluation
- VII. Review**

7 Related Work

相关工作：比较目标、机制与适用条件

- 随机负载均衡：VL2、Monsoon 采用每流 VLB，仍可能发生大流碰撞
- 网络结构：SEATTLE 关注二层扩展；DCell、BCube 探索递归拓扑与多网卡
- 分布式流量工程：TeXCP、MATE、FLARE 的控制粒度和时延假设各有侧重
- 集中式控制：4D、Tesseract 分离控制与转发；Hedera 聚焦大流需求与选路
- Clos 理论：提供分析基础；部署仍需解决商品设备上的多级网络实现问题

- 核心结论：利用**全局需求信息动态调度大流**，可回收静态哈希浪费的路径容量
 - 机制：边缘检测 → 自然需求估计 → 启发式放置 → 交换机规则更新
 - 证据：真实原型与大规模仿真支持吞吐收益；SA 通常更优，但并非所有指标都胜出
 - 边界：收益取决于流速率、持续时间和碰撞程度；短流或主机瓶颈未必受益
 - 启示：同时衡量带宽收益、控制开销与反应时间，并区分实测、仿真和估算